
Transient Execution Attacks

Secure Software and Hardware Systems

Overview

- Introduction
- Microarchitecture
 - Recap: Branch Predictors & Caches
 - Out of order execution
- Transient Execution Attacks
 - Meltdown
 - Spectre
- Defending against microarchitectural attacks
- Recap

Overview

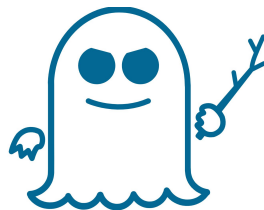
- Introduction
- Microarchitecture
 - Recap: Branch Predictors & Caches
 - Out of order execution
- Transient Execution Attacks
 - Meltdown
 - Spectre
- Defending against microarchitectural attacks
- Recap

Goal of this lecture

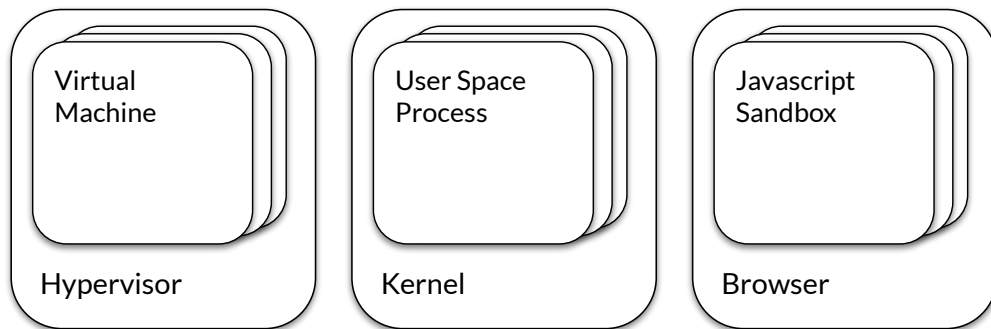
- Show how microarchitectural attacks can leak secrets
- Discuss some of the key TEA
- Discussing potential defenses

Introduction

- We looked at hardware and (cache) timing side channels
 - Both of them subvert leak secrets while program executes normally
- What if we can create those side-channels as attacker?
 - For instance, if we can execute unprivileged code on a target?
- Welcome to transient execution attacks
 - A key technique to leak secrets across privilege domains



Privilege domains?



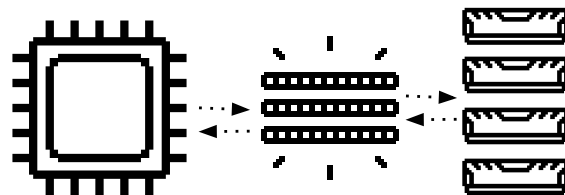
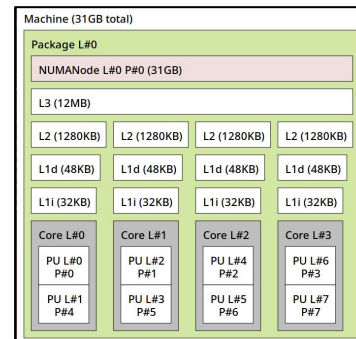
- Modern computers have plenty security domains
 - TAEs allow to attack victims either:
 - Within the same privilege domain (e.g., process A→process B)
 - Across privilege domains (e.g., process→kernel)
-

Overview

- Introduction
- Microarchitecture
 - Recap: Branch Predictors & Caches
 - Out of order execution
- Transient Execution Attacks
 - Meltdown
 - Spectre
- Defending against microarchitectural attacks
- Recap

Recap: Caches

- Caches store recently accessed memory
 - Different types of caches (I-Cache vs. D-Cache)
 - Arranged in a cache hierarchy
 - Tackles a key performance bottleneck: memory in DRAM
- We can build timing side-channels via caches!
 - If a value is in cache, subsequent access will be fast
 - Otherwise, it will be slow



Recap: Branch Predictors

- Branch predictors *predict* potential branch targets
 - Key performance optimizations in modern CPUs
 - Different approaches for direct and indirect branches
- CPU executes predicted branch targets *speculatively*
 - Results are architecturally committed when target turns out correct
 - Otherwise discarded
- Regardless of result, speculative execution leaves trace: microarchitecture
 - This includes the cache(!)

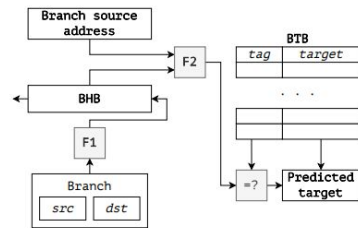
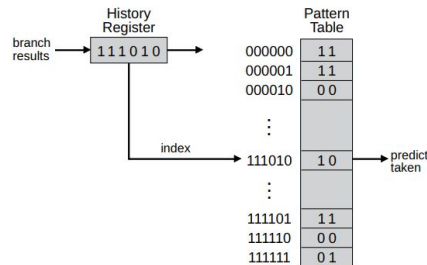


Figure 1: A simplified diagram of indirect branch prediction logic.

Out of Order Execution

- Another key performance optimization, next to speculative execution
 - Hides/minimizes latency (e.g, cache misses, FPU operations, etc)
 - Keeps elements in execution pipeline busy
- Exploits the fact that in μ architecture, instructions can be reordered
 - Usually no issue when there are no data dependencies
 - Results are only committed (architecturally) once all operations completed
- As with speculative execution, leaves data in cache(!)

Live Demo: Out of Order Execution

```
maccess(p1);  
// This will segfault!  
dummy = *NULL;  
maccess(p2);
```

Live Demo: What happened?

- The CPU reordered instructions!

```
maccess(addr1);  
// This will segfault  
dummy = *NULL;  
maccess(addr2);
```

Actual code



```
maccess(addr1);  
maccess(addr2);  
// This will segfault  
dummy = *NULL;
```

Reordered
(transient) code

*THIS FAILS
ARCHITECTURALLY*

*THIS BRINGS ADDR2 INTO
CACHE, ALTHOUGH NEVER
ARCHITECTURALLY
ACCESSED*

Overview

- Introduction
- Microarchitecture
 - Recap: Branch Predictors & Caches
 - Out of order execution
- Transient Execution Attacks
 - Meltdown
 - Spectre
- Defending against microarchitectural attacks
- Recap

Meltdown

In essence, we just saw Meltdown. Now what if:

1. We can (transiently) dereference an interesting value inaccessible to us?
2. We can suppress the architectural failure?
3. Exfiltrate the value?

All of this turns out possible on (older) Intel CPUs:

1. Example: Kernel addresses from user space
2. Use custom SIGSEGV handler (or other methods)
3. Flush+Reload



Meltdown

DECODE ARRAY: ONE ENTRY PER POSSIBLE BYTE VALUE

```
uint8_t decode_arr[256 * 4096];  
flush(decode_arr)  
// This will fault  
x = *kernel_addr;  
maccess(decode_arr[x * 4096]);  
// ...  
// suppress/handle fault  
x = reload(decode_arr)
```

ACCESS INVALID MEMORY

*TRANSIENTLY ENCODE
VALUE IN CACHE*

LEAK DATA FROM CACHE

Suppressing faults: Common Techniques

For Meltdown attacks, we need to *architecturally* suppress faults.

Option 1: Custom segfault handlers

- We just saw this

Option 2: Intel's Transactional Synchronization Extensions (TSX)

- ISA extension for hardware transactional memory
- Key idea: allow parallelism via lock elision
- Data accesses are bundled in transactions
- If transaction fails:
 - Re-roll and try again (HLE)
 - Jmp to custom handler (RTM)
- Now disabled for desktop CPUs, still available on (some) Xeon CPUs

```
xbegin  abort_handler

mov     eax, [rdx]
add     eax, 0x100
mov     [rdx], eax

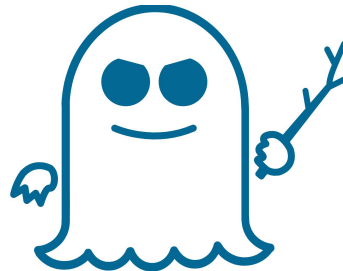
xend
```

Overview

- Introduction
- Microarchitecture
 - Recap: Branch Predictors & Caches
 - Out of order execution
- Transient Execution Attacks
 - Meltdown
 - Spectre
- Defending against microarchitectural attacks
- Recap

Spectre

- Abuses branch (mis)prediction & speculative execution
 - Access sensitive data transiently
 - Cache Side-Channel to exfiltrate data
- Full class of attacks
 - New variants are still discovered
 - Previously thought mitigated issues turn out to be still exploitable
 - Still an active research topic



Spectre-v1 (Spectre-PHT)

```
if (x < array1_size)
    y = array2[array1[x] *
4096];
```

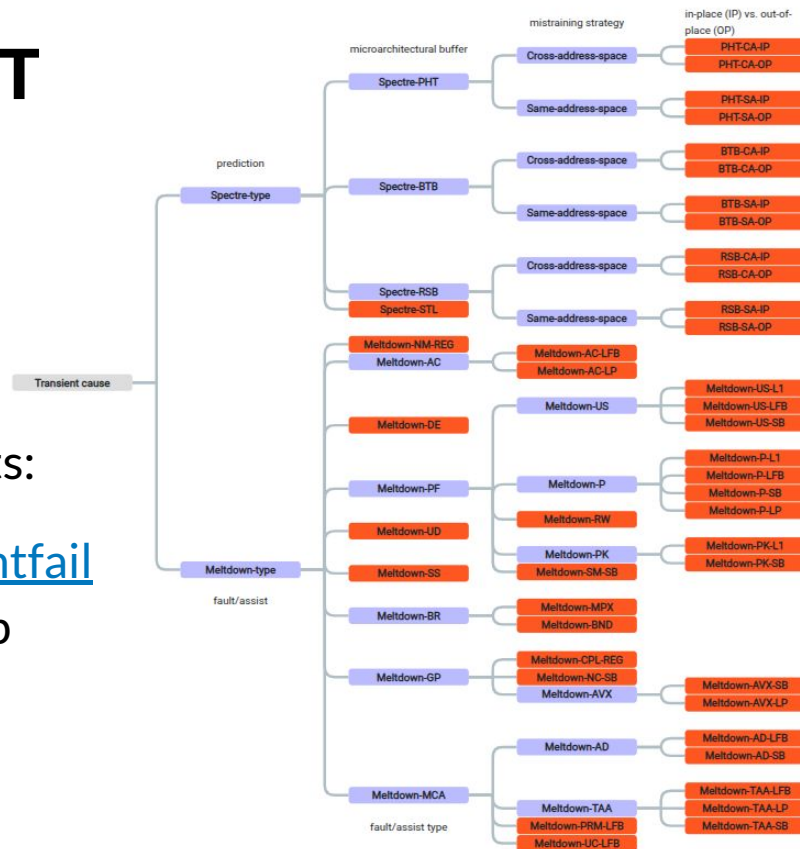
Attack steps:

- 1) Train direct branch predictor to take branch
- 2) Execute target with x out of bounds
 - This will transiently access the value at array1[x]
 - And encode it into array[2]
- 3) Exfiltrate secret via cache-side channel by probing array2
 - E.g., via FLUSH+RELOAD (not shown in code snippet)

Live Demo: Spectre-PHT

Based on the transient fail proof-of-concepts:

- <https://github.com/isec-tugraz/transientfail>
- Selected demo: pocs/spectre/PHT/sa_ip



Spectre Variants & Naming Conventions

Over time, various different variants of spectre reported:

- | | |
|--------------------------------------|-------------------|
| - V1: Bounds Check Bypass (BCB) | - aka Spectre-PHT |
| - V2: Branch Target Injection (BTI) | - aka Spectre-BTI |
| - V3: Rogue Data Cache Load (RDCL) | - aka Meltdown |
| - V4: Speculative Store Bypass (SSB) | - aka Spectre-NG |
| - ????: Return Mispredict | - aka Spectre-RSB |
| - ... | |

But what can these attacks do?

A lot. Examples:

- Derandomize KASLR
- Read data from other processes
- Read data from the kernel
- Bypass hardware security features (e.g., arm PAC)
- Read data from virtual machines on the same physical host
 - Stay for the guest talk for this

More generally: Transient execution attacks break *isolation*

```
[+] Syscall time with large eviction set:  avg: 1008.09  min:  488  max: 2884
[+] Reducing eviction set size: done
[+] Syscall time with small eviction set:  avg: 431.95  min:  396  max: 1742
[+] Required time: 22 seconds
[+] Reload time without eviction:          avg: 11.61  min:   10  max:   13
[+] Reload time    with eviction:         avg: 88.71  min:   72  max:  427
[+] Checking if we can evict all entries:
    - Entry 0: 8 hits (avg time 84.564883)
    - Entry 1: 8 hits (avg time 75.975998)
    OK!
[+] Required time: 0 seconds
[+] Colliding history found after 12777 tries!
[+] Required time: 5 seconds
[+] Breaking KASLR...
[+] Done! page_offset_base = 0xffff8dd900000000
[+] Found /etc/shadow @ 0xffff8dda20945000
[+] Leaking root hash password:
root:$6$T0MlUv90p8yJz7gd53ugl0Zn7EXt4JcNh52F3UGM0TMJuCv8VwXp0zhZs5KfvRMSuy2yA0U135
oFES8x5TBT./8LwFbv0zz8ER#0qu8:19055:0:99990:7:::
[+] Required time: 508 seconds
demo2i7-10700K:-$
[0] 0:spectre-bhb*
*E7-10700K* 17:23 04-Mar-22
```

We have leaked the
root hash password!



But what can these attacks do?

Related paper:

Barberis et al. "Branch History Injection: On the Effectiveness of Hardware Mitigations Against Cross-Privilege Spectre-v2 Attacks" USENIX Security'22

Impact

- Almost every CPU before 2018 vulnerable against Spectre Attacks
 - Software & Microcode mitigations
 - Partially high performance impact
- Newer CPU have hardware mitigations against some variants
 - Fixing Spectre & Meltdown variants took time
 - Meltdown now fully mitigated in hardware
 - Fixing Spectre completely turned out rather difficult

Overview

- Introduction
- Microarchitecture
 - Recap: Branch Predictors & Caches
 - Out of order execution
- Transient Execution Attacks
 - Meltdown
 - Spectre
- Defending against microarchitectural attacks
- Recap

Defenses #1 - Hotfixes and Workarounds

- Intermediate solution after found vulnerabilities
- Either implemented by:
 - Removing particular gadgets used during attacks
 - Changes to software (e.g., adding speculation barriers via FENCE instructions)
 - Microcode updates (e.g., to disable features or change instruction behavior)
- Advantage: Fast mitigation of specific attack vector
- Disadvantages: Performance overhead & may not fully mitigate attacks

Defenses #2 - Hardware Defenses

- Take time to develop
 - CPU design-release cycle takes multiple years
 - As user/typical defender, we have no influence over this
- Sometimes allow to fully remove vulnerability
 - E.g., in-hardware mitigation Meltdown: Checking of privilege before transient memory loads
- May not be complete (for performance reasons)
 - E.g., eIBRS against Spectre-BTB, later bypassed via Spectre-BHI*

* Barberis et al. "Branch History Injection: On the Effectiveness of Hardware Mitigations Against Cross-Privilege Spectre-v2 Attacks" USENIX Security'22

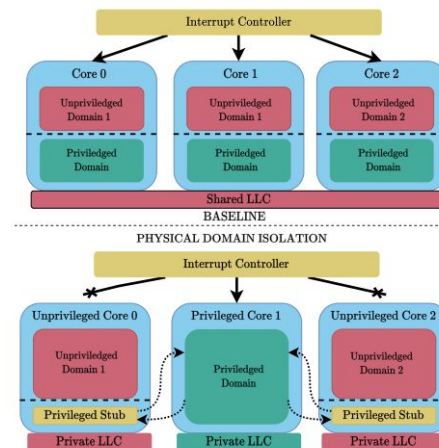
Defenses #3 - Systematic Defenses

Less used in practice:

- Often high performance overhead
- Major changes to software needed

But some examples:

- Secret Free Hypervisor*
- Quarantine**
 - Isolating security domains on different cores
- Undocumented defenses by cloud vendors
 - E.g., aws



Quarantines isolation model.

* <https://www.microsoft.com/en-us/research/wp-content/uploads/2022/07/sf-hypervisor.pdf>

** https://download.vusec.net/papers/quarantine_raid23.pdf

Overview

- Introduction
- Microarchitecture
 - Recap: Branch Predictors & Caches
 - Out of order execution
- Transient Execution Attacks
 - Meltdown
 - Spectre
- Defending against microarchitectural attacks
- Recap

Key Take Away Point

Transient Execution changes the microarchitectural state. Even if results are not used architecturally, transient execution leaves traces in the microarchitecture.

Attackers can abuse this to leak information across privilege domains via side-channels.

Recap

Revisited key microarchitectural components and looked into OOO Execution

- “How” do CPUs actually work?

Looked into Meltdown & Spectre-v1

- “What” are the seminal microarchitectural attacks?

Discussed defense strategies against these attack classes

- “How” do typical defenses against these attacks look like?

Where can I learn more?

Transient execution attacks are a relatively new field. As such, most accurate resources are still academic publications, e.g.:

- The seminal papers in the field: Spectre, Meltdown
- Early follow up works: L1TF/Foreshadow & RIDL/MDS

Beyond this, you can:

- Check various proof of concept attack code on Github
- Stay for the guest lecture ;)

There are also some planned additions to online education platforms

- E.g., pwn.college: <https://pwn.college/system-security/speculative-execution/>
-